

SIMATS ENGINEERING

ASSESSMENT TOOL 2

NAME:S.DEVI SHREE

REGISTER NO:192511149

COURSE NAME: Database Management Systems

COURSE CODE: CSA0503

CO5: Implement database operations using query processing, optimization, and transaction management to ensure consistency and reliability. (BL3, BL4)

Activity Tool : Pseudocode Writing Task (Algorithm Design) (Medium)

Assessment Tool Weightage: 35%

Code of Conduct:

I S.DEVI SHREE (192511149) certify that this submission is myoriginal work and that I have adhered to the guidelines specified for this assessment.I understand that any violation of academic integrity rules will result in disciplinary action.

1. Write pseudocode for the complete query processing pipeline: from SQL input through parsing, optimization, and execution to result output.

Complete Query Processing Pipeline

Aim

To process an SQL query from input until the final result is produced.

Pseudocode

ALGORITHM Query_Processing(SQL_Query)

INPUT:

SQL_Query

OUTPUT:

Query_Result

BEGIN

1. RECEIVE SQL_Query

2. LEXICAL_ANALYSIS:

Break SQL query into tokens.

Identify keywords, table names, column names,

operators, constants, etc.

3. PARSING:

Check whether the SQL syntax is correct.

Construct a Parse Tree / Syntax Tree.

IF syntax error exists THEN

 Display "Syntax Error"

 STOP

END IF

4. SEMANTIC_ANALYSIS:

Check whether tables and columns exist.

Check data types and authorization.

IF semantic error exists THEN

 Display "Semantic Error"

 STOP

END IF

5. QUERY_REWRITING:

Simplify the query.

Apply selection and projection transformations.

Remove unnecessary operations.

6. QUERY_OPTIMIZATION:

Generate different possible execution plans.

Estimate the cost of each plan.

Select the plan having minimum estimated cost.

7. EXECUTION:

Send the selected execution plan to
the query execution engine.

8. ACCESS_DATA:

Read required records from disk/database buffer.

9. PRODUCE_RESULT:

Perform joins, selections, sorting,
grouping and aggregation as required.

10. RETURN Query_Result

END

Pipeline

SQL Query



Lexical Analysis



Parsing



Semantic Analysis



Query Rewriting



Query Optimization



Execution Plan



Execution Engine



Database Access



Result

2. Design a pseudocode algorithm for cost-based query optimization that estimates and compares I/O costs for different join orderings.

Cost-Based Query Optimization

Aim

To select the join order having the lowest estimated I/O cost.

Pseudocode

ALGORITHM Cost_Based_Join_Optimization(Tables)

INPUT:

Tables

Join Conditions

Number of tuples

Number of buffer pages

OUTPUT:

Best_Join_Order

BEGIN

Best_Cost $\leftarrow$ INFINITY

Best_Join_Order $\leftarrow$ NULL

Generate all possible join orderings of Tables

FOR each Join_Order DO

Cost $\leftarrow$ 0

Current_Result $\leftarrow$ first table in Join_Order

FOR each next table R in Join_Order DO

Estimate pages(Current_Result)

Estimate pages(R)

Calculate I/O cost of joining

Current_Result with R

Cost $\leftarrow$ Cost + Estimated_Join_Cost

Update Current_Result statistics

END FOR

```

    IF Cost < Best_Cost THEN
        Best_Cost ← Cost
        Best_Join_Order ← Join_Order
    END IF
END FOR

RETURN Best_Join_Order, Best_Cost
END

```

Simple Nested Loop I/O Cost

For relations R and S:

$$\text{Cost} = B(R) + B(R) \times B(S)$$

where:

$B(R)$ = number of pages in R

$B(S)$ = number of pages in S

Example

Suppose:

R = 100 pages

S = 50 pages

T = 20 pages

Possible orders:

$(R \bowtie S) \bowtie T$

$R \bowtie (S \bowtie T)$

Calculate the estimated cost of both and choose the order with **minimum cost**.

Conclusion

Cost-based optimization selects the execution plan with the lowest estimated I/O cost.

3. Write pseudocode for the Nested Loop Join algorithm and analyze its time complexity in terms of buffer pages and tuple counts.

Nested Loop Join

Aim

To join two relations using the Nested Loop Join algorithm.

Pseudocode

ALGORITHM Nested_Loop_Join(R, S, Join_Condition)

INPUT:

Relation R

Relation S

Join condition

OUTPUT:

Joined relation

BEGIN

FOR each tuple r IN R DO

FOR each tuple s IN S DO

IF Join_Condition(r, s) = TRUE THEN

Add combined tuple (r, s)

to Result

END IF

END FOR

END FOR

RETURN Result

END

Example

FOR each employee E

FOR each department D

IF E.DeptID = D.DeptID

OUTPUT E + D

END IF

END FOR

END FOR

Time Complexity

If:

n = number of tuples in R

m = number of tuples in S

Then:

$$O(n \times m)$$

Buffer Page Analysis

For a block nested-loop join:

$$Cost = B(R) + \left\lceil \frac{B(R)}{M - 2} \right\rceil B(S)$$

where:

- $B(R)$ = pages of R
- $B(S)$ = pages of S
- M = available buffer pages

4. Write a pseudocode algorithm to detect deadlocks in a transaction system using a Wait-For Graph (WFG) cycle detection.

Deadlock Detection Using Wait-For Graph

Aim

To detect deadlocks using a Wait-For Graph (WFG).

Concept

A node represents a transaction.

An edge:

$T_i \rightarrow T_j$

means:

Transaction T_i is waiting for a resource held by T_j .

A cycle in the WFG indicates deadlock.

Pseudocode

ALGORITHM Detect_Deadlock(WFG)

INPUT:

Wait-For Graph WFG

OUTPUT:

Deadlock status

BEGIN

Mark all transactions as UNVISITED

FOR each transaction T DO

IF T is UNVISITED THEN

IF DFS_Cycle_Detection(T) = TRUE THEN

PRINT "Deadlock Detected"

RETURN TRUE

END IF

END IF

END FOR

PRINT "No Deadlock"

RETURN FALSE

END

Cycle Detection

FUNCTION DFS_Cycle_Detection(T)

BEGIN

Mark T as VISITING

FOR each transaction U connected from T DO

IF U is VISITING THEN

RETURN TRUE

END IF

IF U is UNVISITED THEN

IF DFS_Cycle_Detection(U) = TRUE THEN

RETURN TRUE

END IF

END IF

END FOR

Mark T as VISITED

RETURN FALSE

END

Example

T1 → T2

T2 → T3

T3 → T1

There is a cycle:

T1 → T2 → T3 → T1

Therefore:

DEADLOCK DETECTED

Complexity

$$O(V + E)$$

where V = transactions and E = wait-for edges.

5. Design a pseudocode for the Basic Two-Phase Locking protocol including lock request, grant, wait, and release phases.

Basic Two-Phase Locking Protocol

Aim

To maintain serializability using locking.

Two phases:

1. **Growing Phase** – acquire locks.
2. **Shrinking Phase** – release locks.

Once a transaction releases a lock, it cannot obtain a new lock.

Pseudocode

ALGORITHM Basic_2PL(Transaction T)

BEGIN

Phase ← GROWING

FOR each operation O of T DO

IF O is READ(X) THEN

```

    IF Shared_Lock(X) is available THEN
        Grant Shared_Lock(X)
        Perform READ(X)
    ELSE
        WAIT
    END IF

    ELSE IF O is WRITE(X) THEN
        IF Exclusive_Lock(X) is available THEN
            Grant Exclusive_Lock(X)
            Perform WRITE(X)
        ELSE
            WAIT
        END IF
    END IF

END FOR

Phase ← SHRINKING

FOR each lock held by T DO
    RELEASE lock
END FOR

END

```

Important Rule

Growing Phase:

Lock → Lock → Lock

Shrinking Phase:

Unlock → Unlock → Unlock

After the first unlock:

NO NEW LOCK CAN BE ACQUIRED

Advantage

Basic 2PL ensures **conflict serializability**.

6. Write pseudocode for timestamp-based concurrency control using Thomas; Write Rule for handling conflicting read/write operations.

Timestamp Ordering with Thomas' Write Rule

Aim

To control conflicting operations using transaction timestamps.

Each transaction receives:

TS(T)

Each data item maintains:

Read_TS(X)

Write_TS(X)

READ Operation

ALGORITHM Timestamp_Read(T, X)

BEGIN

IF TS(T) < Write_TS(X) THEN

ABORT T

ELSE

Perform READ(X)

Read_TS(X) $\leftarrow$ MAX(Read_TS(X), TS(T))

END IF

END

WRITE Operation with Thomas' Write Rule

ALGORITHM Timestamp_Write(T, X)

BEGIN

IF TS(T) < Read_TS(X) THEN

ABORT T

ELSE IF TS(T) < Write_TS(X) THEN

IGNORE WRITE

// Thomas' Write Rule

ELSE

Perform WRITE(X)

Write_TS(X) $\leftarrow$ TS(T)

END IF

END

Thomas' Write Rule

If:

$TS(T) < Write_TS(X)$

the write is obsolete.

Instead of aborting the transaction, the system:

IGNORE WRITE

This can allow more schedules to execute than basic timestamp ordering.

7. Write a pseudocode algorithm for the ARIES (Algorithm for Recovery and Isolation Exploiting Semantics) recovery technique with Analysis, Redo, and Undo phases.

ARIES Recovery Algorithm

ARIES stands for:

Algorithm for Recovery and Isolation Exploiting Semantics

It has three major phases:

Analysis

↓

Redo

↓

Undo

Pseudocode

ALGORITHM ARIES_Recovery(Log)

INPUT:

Log records

OUTPUT:

Consistent database

```

BEGIN

/* PHASE 1: ANALYSIS */

Read the most recent checkpoint
Identify active transactions
Build Transaction Table
Build Dirty Page Table
Determine:
    Transaction status
    Last LSN
    Dirty pages

/* PHASE 2: REDO */

Find the smallest recovery LSN
FOR each log record from recovery LSN DO
    IF operation needs REDO THEN
        Repeat the logged operation
    END IF
END FOR

/* PHASE 3: UNDO */

Identify incomplete transactions
Add them to Undo list
WHILE Undo list is not empty DO
    Select transaction with largest LSN
    Undo its latest operation
    Write compensation log record
    Move to previous LSN
END WHILE

Write END record for completed transactions
Database ← Consistent State

END

```

ARIES Phases

Phase	Purpose
Analysis	Find active transactions and dirty pages
Redo	Repeat necessary operations
Undo	Roll back incomplete transactions

8. Design pseudocode for Shadow Paging recovery: maintain current and shadow page tables and roll back on failure.

Shadow Paging Recovery

Aim

To recover the database by maintaining two page tables:

Shadow Page Table

Current Page Table

The shadow table points to the stable database pages.

Pseudocode

ALGORITHM Shadow_Paging

BEGIN

 Create Shadow_Page_Table

 Current_Page_Table $\leftarrow$ Copy(Shadow_Page_Table)

 WHILE transaction is running DO

 IF page needs modification THEN

 Create new copy of page

 Modify new page

 Update Current_Page_Table

 to point to new page

 END IF

 END WHILE

 IF COMMIT THEN

```

    Update database root pointer
    Shadow_Page_Table ← Current_Page_Table
    PRINT "Transaction Committed"
ELSE IF FAILURE THEN
    Current_Page_Table ← Shadow_Page_Table
    PRINT "Rollback Completed"
END IF
END

```

Important Idea

Before commit:

Shadow Table → Old Pages

Current Table → Modified Pages

If failure occurs:

Current Table discarded

Shadow Table restored

Therefore, the database returns to the previous consistent state.

9. Write a pseudocode for Immediate Update recovery using UNDO/REDO log processing after a system crash.

Immediate Update Recovery – UNDO/REDO

Aim

In immediate update, database changes can be written to disk **before the transaction commits**.

Therefore, after a crash:

- Committed transactions → **REDO**
- Uncommitted transactions → **UNDO**

Log Format

<T1 START>

<T1, X, old_value, new_value>

<T1 COMMIT>

Pseudocode

ALGORITHM Immediate_Update_Recovery(Log)

INPUT:

Recovery Log

BEGIN

Committed $\leftarrow$ EMPTY

Active $\leftarrow$ EMPTY

/* Find transaction status */

FOR each log record DO

IF record = START(T) THEN

Add T to Active

ELSE IF record = COMMIT(T) THEN

Remove T from Active

Add T to Committed

END IF

END FOR

/* REDO PHASE */

FOR each log record in forward order DO

IF transaction belongs to Committed THEN

REDO operation

Write new value

END IF

END FOR

/* UNDO PHASE */

FOR each log record in reverse order DO

IF transaction belongs to Active THEN

UNDO operation

Restore old value

END IF

END FOR

PRINT "Database Recovered"

END

Example

Suppose:

<T1 START>

<T1, A, 100, 200>

<T1 COMMIT>

<T2 START>

<T2, B, 300, 500>

CRASH

After recovery:

T1 → REDO → A = 200

T2 → UNDO → B = 300

10. Write pseudocode for a database connectivity manager that handles connection pooling, query execution, and exception rollback.

Database Connectivity Manager

Aim

To manage:

- Database connections
- Connection pooling
- Query execution
- Commit
- Rollback
- Exception handling

Pseudocode

ALGORITHM Database_Connection_Manager

INPUT:

SQL_Query

Query_Parameters

OUTPUT:

Query_Result

BEGIN

Initialize Connection_Pool

TRY

/* GET CONNECTION */

Connection ← Get_Connection_From_Pool()

IF Connection = NULL THEN

PRINT "No connection available"

RETURN

END IF

/* START TRANSACTION */

Begin_Transaction(Connection)

/* PREPARE QUERY */

Statement ← Prepare(SQL_Query)

Bind Parameters to Statement

/* EXECUTE */

Result ← Execute(Statement)

/* CHECK RESULT */

IF Execution_Successful THEN

Commit(Connection)

PRINT "Transaction Committed"

ELSE

Rollback(Connection)

PRINT "Transaction Rolled Back"

END IF

/* RETURN CONNECTION */

Return_Connection_To_Pool(Connection)

```

    RETURN Result
CATCH Database_Exception E
    IF Connection exists THEN
        Rollback(Connection)
        Return_Connection_To_Pool(Connection)
    END IF
    PRINT "Database Error"
    PRINT E.Message
FINALLY
    Close Statement if required
END

```

Connection Pool Flow



